Academic Year: 2026-2027
Semester: III
Class / Branch: SE IT
Subject:ADSA
Name of Instructor: Ms. Sneha Dalvi

Name of Student:Nishant Suryarao
Student ID:26204001
Date of Performance:22/09/2026
Date of Submission:22/09/2026

Experiment No.2

Aim: Construct Binary Search Tree for a given sequence of integers and perform Pre-order, Inorder, and Post-order traversal of the constructed tree. Analyse complexities.

Code:

```
#include <stdio.h>

#include <stdlib.h>

// Define the structure for a BST

node struct Node {  int data;  struct

Node* left;  struct Node* right;

};

// Function to create a new node

struct Node* createNode(int value)

{

 struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

newNode->data = value;
```

```
 newNode->left = newNode->right = NULL;
return newNode;

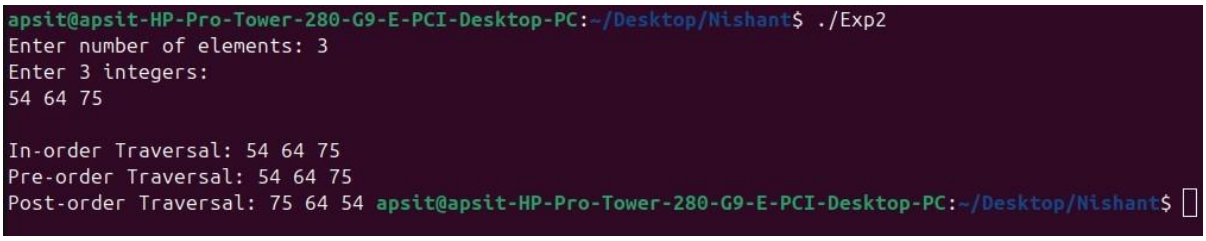
}
// Function to insert a value into the BST struct
Node* insert(struct Node* root, int value) {  if
(root == NULL)
 return createNode(value);if (value < root-
>data)  root->left = insert(root->left, value);
else
 root->right = insert(root->right, value);
return root;
}
// In-order traversal (Left, Root,
Right) void inorder(struct Node*
root) {  if (root != NULL) {
inorder(root->left);  printf("%d ",
root->data);  inorder(root->right);
 }
}
```

```
// Pre-order traversal (Root, Left,
Right) void preorder(struct Node*
root) {  if (root != NULL) {  printf("%d ",
root->data);  preorder(root->left);
preorder(root->right);
 }
}
// Post-order traversal (Left, Right,
Root) void postorder(struct Node*
root) {  if (root != NULL) {
postorder(root->left);  postorder(root-
>right);  printf("%d ", root->data);
 }} int main() {  struct Node* root =
NULL;  int n, value;  printf("Enter
number of elements: ");
scanf("%d", &n);  printf("Enter %d
integers:\n", n);  for (int i = 0; i < n;
i++) {  scanf("%d", &value);  root =
insert(root, value);
 }
```

```
 printf("\nIn-order Traversal: ");
inorder(root);  printf("\nPre-
order Traversal: ");
preorder(root);

 printf("\nPost-order Traversal:
");  postorder(root); return 0;
}
```

Output Screen Shots:



Conclusion: In this experiment, we successfully constructed a Binary Search Tree and performed Pre-order, In-order, and Post-order traversals. We understood how BST insertion and recursive traversal work, and learned that Inorder traversal gives the elements in sorted order.

Experiment 2: Binary Search Tree (BST) Traversals

Definition: A Binary Search Tree is a binary tree in which the left subtree contains smaller values and the right subtree contains greater values than the root.

Algorithm:

1. Start with an empty BST.
2. Read the given integer values.
3. Insert each value according to BST properties.
4. Perform Inorder Traversal: Left → Root → Right.

5. Perform Preorder Traversal: Root → Left → Right.
6. Perform Postorder Traversal: Left → Right → Root.
7. Display all traversal results.
8. Stop.

Conclusion: Thus, a Binary Search Tree was successfully constructed, and its Inorder, Preorder, and Postorder traversals were performed. Inorder traversal displays the elements in sorted order.

Complexity: Average insertion $O(\log n)$; worst case $O(n)$. Each traversal takes $O(n)$.